MATCHREAD · SPRINT DIRECTIVE · INTERNAL

Four Priorities to the US Open

Reconciliation, event-date semantics, eligibility, and the consumer surface filter. **Directive 2.1 — supersedes the priority order in Playbook v2.0.**

Subtraction is the feature. The public product is ATP and WTA 250 and above; everything beneath it may be ingested and must not be surfaced. This directive states the four things that must be true before the US Open draw is published, the code that makes each true, and the evidence that proves it.

DIRECTIVE	2.1 · 21 August 2026 · supersedes Playbook v2.0 §2, §3, §5, §13
ADDS	§3 Event-date and lock-time semantics — new, and the highest-risk item on the list
IMMOVABLE	US Open main draw published Thu 27 Aug; first ball Sun 30 Aug, 11:00 ET
REHEARSAL	Winston-Salem. Its dates must be corrected before it opens, not during.
PROOF WINDOW	Closes with the Cincinnati finals — the stalled branches are the only live evidence available.
VERSION NOTE	§7 is an ordered sequence, not a calendar. Order is the instruction; dates are not.

Green before Winston-Salem opens. The rehearsal depends on it.

Green before the US Open draw is published.

Deliberately deferred. Recorded so it is not rediscovered as an idea.

> THE DISTINCTION THIS DIRECTIVE TURNS ON
>
> **§4 stops the system creating brackets it cannot support. §5 stops the user ever having to think about tournaments MatchRead does not support.** These are different guarantees with different failure modes, they are enforced in different layers, and shipping one is not shipping the other.

01 · DIRECTIVE

The four priorities, and what each one prevents

Playbook v2.0 listed ten priorities. This directive collapses the pre-launch set to four and fixes their order. Everything else in v2.0 stands and moves behind these.

#	PRIORITY	PREVENTS	GREEN BY
1	`Authoritative reconciliation`	A missed provider event stalling a bracket branch indefinitely. Proven by repairing both broken Cincinnati branches with no manual database edit.	REHEARSAL
2	`Event-date and lock-time semantics`	A bracket that locks after play has begun. The only item here whose failure cannot be repaired after the fact.	REHEARSAL
3	`Database eligibility`	The *system* creating a bracket on a tournament MatchRead has not committed to supporting. Enforced at write time, in Postgres.	USO DRAW
4	`Consumer surface hard filter`	The *user* encountering a tournament MatchRead has not committed to supporting. Enforced at read time, in the query.	USO DRAW

1.1 Why 3 and 4 are not one task

They protect different parties against different failures, and either can pass while the other fails.

	§4 · DATABASE ELIGIBILITY	§5 · HOMEPAGE FILTER
`Protects`	The system's integrity	The user's attention
`Enforced at`	Write time — insert policy on `brackets`	Read time — the calendar query itself
`Layer`	Postgres. Survives a stale client, a cached page, a direct API call.	The server component's data fetch. Never a client-side `.filter()` over a wider result set.

	§4 · DATABASE ELIGIBILITY	§5 · HOMEPAGE FILTER
Failure if absent	A Challenger bracket exists, has members, has picks, and cannot be settled to the product's standard. Data you must then decide what to do with.	A user browses to Cancun Challenger, fills a bracket, and discovers the product's boundary by hitting it. Trust cost, not data cost.
Test	`insert into brackets` against an ineligible draw is rejected by the database.	The homepage response contains zero rows with `tier not in (…)`, asserted on the payload, not the DOM.

> **RULE**
>
> The homepage query **filters**, it does not sort. An ineligible tournament is not returned, ranked last, or hidden with CSS. `featured` decides prominence *within* the eligible set and never decides membership of it.

02 · PRIORITY 1 · BEFORE WINSTON-SALEM

Authoritative reconciliation

Two Cincinnati branches are stalled. They look like the same defect and they are not, and the difference determines whether your sweep finds them.

BRANCH	SYMPTOM	DIAGNOSIS
ATP · Fonseca–O'Connell R32, 18 Aug	Timestamp present, no winner	The fixture row exists. The result never arrived, or arrived without a winner ID. A stored-row diff will find this one.
WTA · Wang–Svitolina R32	No timestamp, no winner	**The fixture row may not exist at all.** A stored-row diff will never find it, because you cannot diff a row you never had.

2.1 The design decision that determines whether this works

```
// WRONG. Repairs the ATP branch, never sees the WTA one.
// The loop's domain is your data, so a fixture you never ingested is
// invisible to it — for ever, by construction.
const stored = await repo.getFixturesByTournament(tournamentId);
for (const fixture of stored) {
  const remote = await provider.getFixture(fixture.providerFixtureId);
  if (remote.winnerId !== fixture.winnerId) await repair(fixture, remote);
}
```

```
// RIGHT. The provider's set is the domain. Your rows are the comparison.
// Missing rows are created; changed rows are repaired; extra rows are flagged.
export async function reconcileTournament(
  tournamentId: TournamentId,
  runId: RunId,
): Promise<ReconcileSummary> {
  const tournament = await repo.getTournament(tournamentId);
  const remote = await provider.getTournamentFixtures(tournament);  // authoritative set
  const stored = await repo.getFixturesByTournament(tournamentId);
  const byProviderId = new Map(stored.map(f => [f.providerFixtureId, f]));

  const summary = { created: 0, repaired: 0, settled: 0, orphaned: 0 };

  for (const raw of remote) {
    const next = normalizeFixture(raw);                // packages/core/src/ingest
    const current = byProviderId.get(next.providerFixtureId) ?? null;
    byProviderId.delete(next.providerFixtureId);

    if (!current) summary.created++;
    else if (!needsRepair(current, next)) continue;
    else summary.repaired++;

    const saved = await repo.upsertFixture(next);      // keyed on providerFixtureId
    await audit.logRepair({ runId, tournamentId, before: current, after: saved });

    if (canAdvanceWinner(saved.outcome, saved.winnerPlayerId)) {
      const result = await settleDraw({ fixtureId: saved.id, runId });
      if (result.applied) summary.settled++;
    }
  }

  // Anything left in the map is a row we hold that the provider does not.
  // Never delete it silently — a stored fixture with no provider counterpart is
  // either a tour-identity defect or a provider outage, and both need a human.
  for (const orphan of byProviderId.values()) {
    summary.orphaned++;
    await audit.logOrphan({ runId, tournamentId, fixture: orphan });
  }

  return summary;
}
```

2.2 Find the missing row before you build the sweep

Ten minutes of diagnosis will tell you whether the WTA branch is a missing row or a null-winner row, and that determines how much of the above you actually need this week.

```
-- Seats whose match should have resolved but whose fixture is absent.
-- If this returns the Wang–Svitolina seat, the row was never created.
select d.tournament_id, s.round, s.position, p.display_name
from draw_seats s
join draws     d on d.id = s.draw_id
join players   p on p.id = s.player_id
left join fixtures f
  on f.draw_id = s.draw_id
 and f.round   = s.round
 and s.position between f.position_lo and f.position_hi
where d.tournament_id = :tournament_id
  and f.id is null
order by s.round, s.position;

-- Fixtures that exist, are past, and have no winner. The ATP branch shape.
select f.id, f.provider_fixture_id, f.round, f.starts_at, f.outcome
from fixtures f
join draws d on d.id = f.draw_id
where d.tournament_id = :tournament_id
  and f.winner_player_id is null
  and (f.starts_at < now() - interval '6 hours' or f.starts_at is null)
order by f.round, f.starts_at nulls first;
```

2.3 Idempotency, before this touches production

Cincinnati is mid-tournament with most of the draw already settled. A sweep that is not replay-safe turns a two-branch defect into a whole-draw one. Guard at the settlement boundary, not in the caller.

```
-- Settlement is claimed, not requested. The unique index is the lock.
create table if not exists settlement_claims (
  fixture_id   uuid primary key references fixtures(id),
  outcome      match_outcome not null,
  winner_id    uuid not null references players(id),
  claimed_at   timestamptz not null default now(),
  run_id       uuid not null
);

-- Re-running with the same result is a no-op. Re-running with a DIFFERENT
-- winner is a genuine correction (an overturned retirement, a corrected feed)
-- and must unwind the prior advancement rather than stack on top of it.
create or replace function claim_settlement(
  p_fixture uuid, p_outcome match_outcome, p_winner uuid, p_run uuid
) returns text language plpgsql as $$
declare existing settlement_claims;
begin
  select * into existing from settlement_claims where fixture_id = p_fixture;
  if not found then
    insert into settlement_claims values (p_fixture, p_outcome, p_winner, now(), p_run);
    return 'claimed';
  elsif existing.winner_id = p_winner and existing.outcome = p_outcome then
    return 'noop';
  else
    return 'conflict';   -- caller unwinds, then re-claims. Alerts.
  end if;
end $$;
```

2.4 The proof protocol

"Cincinnati heals without a manual DB edit" is the right standard. These five gates are what make it mean what you want it to mean. All five, or the proof does not count.

	GATE	EVIDENCE
1	Scheduled, not invoked	The repair appears in the audit log with a `run_id` belonging to a `pg_cron` run you did not trigger. Hand-running the job proves the code and not the system, and cron being armed is one of the four open preflight questions.
2	Both branches	ATP R32 and WTA R32 both resolve, and the resolution propagates through R16, QF and SF without further intervention. If only the ATP branch heals, §2.1 is why.
3	Audited	Every repair carries tournament ID, fixture ID, before state, after state, provider timestamp and run ID. A branch that heals with no audit row healed by accident.
4	Replay-safe	Run the sweep three more times. `settled` is zero on every subsequent run and no `bracket_snapshots` row changes.
5	Detection, not just repair	**Withhold a fourth result deliberately** and confirm the sweep finds it unprompted. Gates 1–4 prove you can fix a break you already knew about. Only this one proves the system would have caught it without you.

> **WINDOW**
>
> Cincinnati's finals are Sunday 23 August. After that the live data you are proving against is gone, and Winston-Salem has to serve as both the reconciliation proof and the rehearsal — one event doing two jobs, with the US Open draw four days behind it.

`Tests · packages/settlement/src/reconcile.test.ts · supabase/tests/25_reconciliation.test.sql`

03 · PRIORITY 2 · BEFORE WINSTON-SALEM

Event-date and lock-time semantics

This is the only item in this directive whose failure cannot be repaired afterwards. A stalled branch heals whenever the sweep is right. A bracket that locked after the first ball has already let people pick winners they had watched.

3.1 The finding

Three events, three different relationships between the displayed date and the actual start of the main draw.

EVENT	SITE SHOWS	MAIN DRAW	EVIDENCE
US Open	**31 Aug**	**30 Aug**	Main draw begins Sunday 30 August and ends with the men's final on Sunday 13 September; qualifying runs 24–27 August; the draw ceremony is Thursday 27 August. The site's end date, 13 Sep, is already correct — and 31 Aug to 13 Sep is fourteen days, not the fifteen the format has.
Winston-Salem	**24 Aug**	**23 Aug** (expected)	Every recent edition runs Sunday to Saturday — 2025 was 17–23 August, 2024 was 18–24. With the Slam on the 30th, 2026 should be Sunday 23 to Saturday 29. **Verify against the provider before the rehearsal;** this one is inferred from pattern, not confirmed.
Cincinnati	**10 Aug**	**13 Aug**	Its own R128 fixtures run 13–14 August. The displayed start is three days *earlier* than the main draw, which looks like a qualifying date.

> DIAGNOSIS
>
> The errors point in opposite directions, so this is not an off-by-one. It is a missing concept: `starts_on` **does not mean "main draw begins," and no column does.** Whatever the provider supplies is being written into one ambiguous date field and then read as though it meant the thing the product depends on. Correcting three rows would leave the defect in place for the next tournament.

3.2 Probe first, in one sitting

Fetch the raw provider payload for all three events and print the date fields beside the stored values. The answer is in that diff and nowhere else — do not reason about it from the rendered page.

```
$ pnpm tennis:probe-dates --events atp-21349,wta-16743,atp-21348,atp-21347

event        provider field        raw value      stored starts_on   main draw
atp-21349    ?                     ?              2026-08-31          2026-08-30
wta-16743    ?                     ?              2026-08-31          2026-08-30
atp-21348    ?                     ?              2026-08-24          2026-08-23 ?
atp-21347    ?                     ?              2026-08-10          2026-08-13

# Hypotheses to falsify, in order:
#   1. The provider exposes a week-commencing Monday, not a start date.
#   2. The provider exposes both qualifying and main-draw dates and the
#      importer takes whichever appears first in the payload.
#   3. A timestamp is being cast to a date in the wrong zone, moving a
#      Sunday evening start into Monday.
#   4. Different endpoints disagree and different code paths read each.
```

3.3 Schema: name the dates you actually depend on

```
-- 0032_event_dates.sql
-- Renaming rather than adding, deliberately: the compiler then finds every
-- reader of the ambiguous field. Costs a database.types.ts regeneration and a
-- sync to apps/web and apps/mobile — cheap, and it is the point of doing it.

alter table tournaments rename column starts_on to provider_week_starts_on;
comment on column tournaments.provider_week_starts_on is
  'Raw provider anchor. Audit and diagnosis only. NEVER read by product code.';

alter table tournaments
  add column main_draw_starts_on  date,
  add column qualifying_starts_on date,
  add column ends_on              date,
  add column draw_published_at    timestamptz,
  add column time_zone            text,      -- IANA, e.g. America/New_York
  add column locks_at             timestamptz;

-- The product contract, asserted by the database rather than by convention.
alter table tournaments
  add constraint tournaments_dates_ordered check (
    (qualifying_starts_on is null or main_draw_starts_on is null
      or qualifying_starts_on <= main_draw_starts_on)
    and (ends_on is null or main_draw_starts_on is null
      or main_draw_starts_on <= ends_on)
  ),
  add constraint tournaments_lock_within_event check (
    locks_at is null or main_draw_starts_on is null
    or locks_at <= (main_draw_starts_on + interval '1 day')
  );

-- Eligible tournaments must carry a zone. Without one, every displayed time
-- falls back to UTC, which is the defect currently on every draw page.
create index tournaments_missing_zone
  on tournaments (id) where bracket_eligible and time_zone is null;
```

3.4 Lock is an instant, and it is never derived from a date

`starts_on` could not be the lock source even when it is correct, because a date is not a time. Midnight on the day of play is not the same moment as the first ball, and the gap between them is a day session.

```
// packages/core/src/tournament/lock.ts
//
// locks_at is SET, not COMPUTED. It is sourced from the published order of
// play, which appears the evening before, and it is written once. This
// function exists to VALIDATE it, never to generate it.

export function assertLockIsSound(t: Tournament, firstFixture: Fixture | null): void {
  invariant(t.locksAt !== null,
    `${t.slug}: locks_at is null — a draw cannot open without a lock instant`);
  invariant(t.timeZone !== null,
    `${t.slug}: time_zone is null — lock would render in UTC`);
  invariant(t.drawPublishedAt !== null && t.drawPublishedAt < t.locksAt,
    `${t.slug}: lock precedes draw publication — the entry window is empty`);

  // The one that matters. Once any main-draw fixture has a start time, the
  // lock must precede it. This is the assertion that would have caught a
  // Monday-anchored US Open before a single bracket was submitted.
  if (firstFixture?.startsAt) {
    invariant(t.locksAt <= firstFixture.startsAt,
      `${t.slug}: lock ${t.locksAt.toISOString()} is after first ball ` +
      `${firstFixture.startsAt.toISOString()} — brackets would close mid-play`);
  }
}
```

3.5 The two events, concretely

```
-- US Open 2026. Main draw Sunday 30 August; day sessions from 11:00 ET,
-- Arthur Ashe from 12:00 ET. Lock at first ball of the main draw.
--
-- ⚠ The draw ceremony DATE is Thursday 27 August and is sourced. The HOUR
--   below is an assumption and must be verified before this runs — a wrong
--   draw_published_at silently shortens or lengthens the stated entry window
--   on every surface that renders it. Verify, or leave it null and let
--   assertLockIsSound refuse to open the draw until someone knows.
update tournaments set
  main_draw_starts_on  = date '2026-08-30',
  qualifying_starts_on = date '2026-08-24',
  ends_on              = date '2026-09-13',
  draw_published_at    = timestamptz '2026-08-27 12:00:00-04',  -- ⚠ VERIFY
  time_zone            = 'America/New_York',
  locks_at             = timestamptz '2026-08-30 11:00:00-04'   -- 15:00Z
where provider_tournament_id in ('21349', '16743');

-- Winston-Salem. VERIFY against the ATP calendar before writing; the
-- 23 August start is inferred from the Sunday-to-Saturday pattern of prior
-- editions, not confirmed from a source.
update tournaments set
  main_draw_starts_on = date '2026-08-23',
  ends_on             = date '2026-08-29',
  time_zone           = 'America/New_York'
where provider_tournament_id = '21348';
```

> **CONSEQUENCE WORTH PLANNING AROUND**
>
> The US Open draw ceremony is Thursday 27 August and the first ball is Sunday the 30th. **Your entire entry window is about three days.** That is the funnel — every invite, every league formed, every bracket filled. Playbook v2.0 §12 describes the traffic as a cliff rather than a ramp; this is how short the cliff is.

`Tests · supabase/tests/27_event_dates.test.sql · packages/core/src/tournament/lock.test.ts`

04 · PRIORITY 3 · BEFORE THE US OPEN DRAW

Database eligibility

A tournament is bracket-eligible when it is ATP or WTA and its tier is 250 or above. Five enum values qualify. Everything else may be ingested, rated against, and searched — and may not carry a bracket.

ELIGIBLE TIER	ENUM VALUE	COVERS
Grand Slam	`grand_slam`	All four, both tours
Tour Finals	`tour_finals`	ATP Finals, WTA Finals
1000	`masters_1000`	ATP Masters 1000 and WTA 1000 — level, not tour
500	`tour_500`	ATP 500 and WTA 500
250	`tour_250`	ATP 250 and WTA 250

`challenger`, `wta_125`, `itf` and `other` are ineligible. `ingestion_enabled` stays `true` for them: the rating engine, player search and future coverage all benefit, and none of it reaches a bracket.

4.1 One definition, in SQL, mirrored in TypeScript, asserted equal

```
-- 0033_bracket_eligibility.sql
-- Enum values are added in 0032 and used here: Postgres will not permit both
-- in one transaction, and Supabase wraps each migration in one.

-- Subtraction only. There is deliberately no force_on: see the note below.
create type product_override as enum ('force_off');

alter table tournaments
  add column product_override  product_override,          -- null = policy decides
  add column ingestion_enabled boolean not null default true,
  add column featured          boolean not null default false;

create or replace function is_bracket_product(
  p_tour     tour,
  p_tier     tournament_tier,
  p_override product_override default null
) returns boolean language sql immutable as $$
  select case
    when p_override = 'force_off' then false
    when p_tour not in ('atp', 'wta') then false
    else p_tier in ('grand_slam', 'tour_finals', 'masters_1000', 'tour_500', 'tour_250')
  end
$$;

-- Stored and generated: the calendar query becomes an index scan, and the
-- flag cannot drift from the policy that defines it.
alter table tournaments
  add column bracket_eligible boolean
  generated always as (is_bracket_product(tour, tier, product_override)) stored;

create index tournaments_eligible_by_date
  on tournaments (main_draw_starts_on) where bracket_eligible;
```

```
// packages/core/src/tournament/eligibility.ts
// The TypeScript mirror. 23_eligibility.test.sql feeds this exact set into the
// SQL function and asserts agreement — that assertion is the thing that stops
// the two definitions drifting the first time someone adds a tier.

export const PUBLIC_TIERS = [
  'grand_slam', 'tour_finals', 'masters_1000', 'tour_500', 'tour_250',
] as const satisfies readonly TournamentTier[];

export const PUBLIC_TOURS = ['atp', 'wta'] as const satisfies readonly Tour[];

export function isBracketProduct(t: {
  tour: Tour | null;
  tier: TournamentTier | null;
  productOverride?: 'force_off' | null;
}): boolean {
  if (t.productOverride === 'force_off') return false;   // subtraction only
  if (!t.tour || !PUBLIC_TOURS.includes(t.tour)) return false;
  if (!t.tier) return false;
  return PUBLIC_TIERS.includes(t.tier);
}
```

4.2 Enforce it at write time

This is the half of the guarantee that survives a stale client. Lock is already enforced by the trigger in migration `0013`; this policy adds eligibility and draw publication to the same insert path.

```
-- 0033 continued. Belt and braces: the policy refuses the insert, and the
-- constraint refuses it even from a service-role context that bypasses RLS.

create policy brackets_insert_eligible_only
  on brackets for insert to authenticated
  with check (
    exists (
      select 1
      from draws d
      join tournaments t on t.id = d.tournament_id
      where d.id = brackets.draw_id
        and t.bracket_eligible
        and d.status = 'published'
        and now() < d.locks_at
    )
  );

-- Denormalized guard column, maintained by trigger from the parent draw, so
-- the check is available to a table constraint. A subquery cannot be.
-- No default. A default of true is a permission the caller never asked for,
-- and it is the value that would be written if the trigger below were ever
-- dropped, disabled, or bypassed by a copy-paste insert.
alter table brackets add column tournament_eligible boolean;

create or replace function brackets_stamp_eligibility() returns trigger
language plpgsql as $$
begin
  select t.bracket_eligible into new.tournament_eligible
  from draws d join tournaments t on t.id = d.tournament_id
  where d.id = new.draw_id;

  if new.tournament_eligible is null then
    raise exception 'bracket on draw %: no tournament, or eligibility unknown',
      new.draw_id;
  end if;
  return new;
end $$;

create trigger brackets_stamp_eligibility_trg
  before insert on brackets
  for each row execute function brackets_stamp_eligibility();

-- Stamp the rows that predate the trigger, then constrain. If the backfill
-- leaves a null, an existing bracket points at a draw with no tournament and
-- that is a separate defect — stop and look at it rather than defaulting it.
update brackets b set tournament_eligible = t.bracket_eligible
from draws d join tournaments t on t.id = d.tournament_id
where d.id = b.draw_id and b.tournament_eligible is null;

alter table brackets
  alter column tournament_eligible set not null,
  add constraint brackets_require_eligible check (tournament_eligible);
```

WHY THERE IS NO FORCE_ON

An override that can promote an arbitrary row into the public product defeats the boundary it sits inside — and it will be reached for at 2am, by whoever is on call, under exactly the conditions in which nobody is checking whether the draw order is right. The one legitimate use for promotion is a deliberate expansion of coverage, and that is a tier-level product decision: it belongs in `PUBLIC_TIERS`, reviewed, tested and shipped. **The override can only subtract.**

DEMOTION NEVER DESTROYS DATA

Setting `bracket_eligible` to false — or `product_override` to `force_off` as an incident action — removes a tournament from discovery and from new entry. **It does not touch existing brackets.** A bracket filled before demotion stays readable by its owner and its league, keeps its picks, and still settles. Assert this explicitly: it is the rule most likely to be broken by a well-meaning cascade delete.

Tests · supabase/tests/23_eligibility.test.sql

05 · PRIORITY 4 · BEFORE THE US OPEN DRAW

Consumer surface hard filter

One predicate, every consumer surface, no exceptions. Cancun Challenger, Prague Challenger, every WTA 125 and every ITF event disappear from all of them completely — not sorted below, not collapsed, not dimmed. Absent.

SECTION	CONTAINS	JOB
`Open for picks`	Eligible · draw published · before lock	The only section with the primary CTA. If it is empty the page says so and names what is next.
`Upcoming`	Eligible · draw pending, ordered by `featured desc, main_draw_starts_on asc`	Anticipation. A league can be assembled before the draw lands.
`Live`	Eligible · locked or on court	Follow your picks. Secondary CTA only — never "Fill a bracket."

5.1 Every surface, not just the homepage

A boundary that holds on one page and not the next is not a boundary. Each of these is a way a user reaches a tournament, and each one needs the same predicate.

SURFACE	BEHAVIOUR	NOTE
`Homepage sections`	ABSENT	Open for picks · Upcoming · Live. §5.2.
`/tournaments calendar`	ABSENT	The larger browse. Today it still lists six Challengers; it is the same query.
`/tournaments/[ref] detail`	404	An ineligible tournament has no public page. Not a stub, not a "coming soon" — the route returns not-found, so a shared link cannot become an entry point.

SURFACE	BEHAVIOUR	NOTE
Surface filter chips	DERIVED	Rendered from the surfaces present in the eligible set. A chip that returns nothing teaches a user the product is empty.
Search and autocomplete	ABSENT	Player search is unaffected — players from every tier remain searchable. Tournament search is bounded.
League tournament picker	ABSENT	The commissioner flow at `/leagues/new?tournament=`. This is the one that would otherwise create the unsupported bracket §4 exists to refuse — belt to §4's braces.
Sitemap · OG · feeds	ABSENT	Anything that puts a URL in front of a crawler. An indexed Challenger page is a permanent entry point into a surface the product does not support.
League-scoped bracket routes	UNAFFECTED	**The exception, and it matters.** A bracket filled before a tournament was demoted stays readable at `/leagues/[slug]/t/[ref]` and still settles. Eligibility governs discovery and new entry; it never reaches into a league someone already built.

5.2 The query

```
-- A view, so the predicate exists once and every reader inherits it.
-- There is no code path to the homepage that can omit `where bracket_eligible`.
create or replace view public_calendar as
select
  t.id, t.slug, t.name, t.tour, t.tier, t.surface, t.environment,
  t.main_draw_starts_on, t.ends_on, t.time_zone, t.featured,
  d.id as draw_id, d.status as draw_status, d.locks_at,
  exists (select 1 from fixtures f
          where f.draw_id = d.id and f.starts_at <= now()) as any_match_started
from tournaments t
left join draws d
  on d.tournament_id = t.id and d.kind = 'singles'
where t.bracket_eligible;                -- ← the whole product boundary, once

comment on view public_calendar is
  'The only source for consumer tournament browse. Ineligible tournaments are '
  'not present. Do not add a tier or tour predicate downstream of this view, '
  'and do not read `tournaments` directly from a consumer surface.';
```

```
// apps/web/src/data/calendar.ts
// Sectioning happens on an already-filtered set. There is no `.filter(eligible)`
// anywhere in this file, because nothing ineligible ever arrives.

export type HomeSection = 'open' | 'upcoming' | 'live';

export function sectionFor(t: PublicTournament, now: Date): HomeSection | null {
  if (t.drawStatus === 'complete') return null;
  if (t.anyMatchStarted || (t.locksAt && now >= t.locksAt)) return 'live';
  if (t.drawStatus === 'published') return 'open';
  return 'upcoming';                                        // draw pending
}

export async function getHomepage(now = new Date()): Promise<Homepage> {
  const rows = await db
    .from('public_calendar')                                // filtered at source
    .select('*')
    .gte('ends_on', isoDate(now))
    .order('featured', { ascending: false })
    .order('main_draw_starts_on', { ascending: true });

  const sections: Homepage = { open: [], upcoming: [], live: [] };
  for (const row of rows) {
    const section = sectionFor(row, now);
    if (section) sections[section].push(row);
  }
  return sections;
}
```

THREE RULES THAT KEEP THIS FROM ERODING

No consumer surface reads `tournaments` directly. Every surface in §5.1 reads `public_calendar`. Add a CI grep for `from('tournaments')` across `apps/web/app/(public)`, the league creation flow and the sitemap route — the same way the service-role key is policed. **`featured` ranks, it never admits.** The moment `featured` can pull an ineligible row into view, the boundary is a suggestion. **A new consumer surface inherits the predicate or it does not ship.** The next tournament-facing page somebody adds is the one that will reintroduce this, so the test in §7 asserts on the payload of every route in §5.1 rather than on the homepage alone.

5.3 The empty state is load-bearing

With Challengers filtered out, there is a stretch between the Cincinnati finals and the US Open draw in which every eligible event — Winston-Salem, Monterrey, Cleveland, both US Opens — is draw pending. **Nothing is open, for several days.** Anyone arriving from search or a shared link meets the empty state, and it has to carry the product on its own.

```
// packages/i18n/src/en/calendar.ts — and the es/ja equivalents, or it
// renders as a key. The rule from the Phase 4 handoff: say what is true, then
// say what happens next. "No tournaments open" does the first half only.

export const calendar = {
  openEmpty: {
    heading: 'Nothing open right now.',
    body: (event: string, when: string) =>
      `The ${event} draw lands ${when}. Your league can be ready before it does.`,
  },
  openEmptyNoNext: {
    heading: 'Nothing open right now.',
    body: 'The next draw is a few days out. We will have it the moment it lands.',
  },
};
```

The `event` and `when` arguments come from the first row of `upcoming` — which is why Upcoming must be queried even when Open is empty, and why the empty state is a component with data rather than a string.

Tests · apps/web/tests/homepage-filter.test.ts · apps/web/e2e/calendar-boundary.spec.ts

---

06 · DEFERRED BY DECISION

The coverage surface, and when to build it

Recorded so it is not rediscovered as a new idea in October, and so nobody reads the filter in §5 as a judgement that Challenger tennis does not matter.

When MatchRead deliberately expands into coverage, the surface is two visually distinct tiers rather than one longer list:

TIER	TREATMENT
Bracket events	ATP and WTA 250+. Full treatment, primary CTA, the court hairline, the Daily Check.
More tennis	Challenger, WTA 125, ITF. A quieter secondary section. Results and player pages, no bracket affordance, no CTA. Reachable, never promoted.

Trigger conditions, all three: **(a)** the integrity gate has run green across a full 250+ season without manual intervention; **(b)** reconciliation cadence has been measured against provider quota at 250+ volume, with headroom to spare; **(c)** there is evidence users want it — search queries, support requests, or league members asking for an event that is not there.

WHY THE ORDERING IS RIGHT

Guaranteeing data quality across 250 simultaneous events and guaranteeing it across ten are different engineering problems. The public trust surface should be the size the team can actually defend, and it should grow only when defending it has stopped being interesting. The ingestion engine keeps learning from everything underneath in the meantime — nothing is thrown away, it is only not promised.

07 · SEQUENCE

Order of work

No dates. The sequence is what matters: each row depends on the one above it, and the two read-only investigations at the top change what the rest of the work looks like. Do them first even if nothing else gets done today.

	DELIVERABLE	DONE WHEN
1	`Date probe · §3.2` `Fixture diagnosis · §2.2`	You know what the provider actually returns for all four events, and whether the WTA Cincinnati branch is a missing row or a null winner. Both are read-only and can run in the same sitting. **Everything below is shaped by the answers.**
2	`Date schema and lock instants · §3.3–3.5`	Migration applied. Every eligible tournament carries a main-draw date, an IANA zone and an explicit `locks_at`. `assertLockIsSound` passes for the US Open and fails for a deliberately Monday-anchored fixture.
3	`Idempotency guard · §2.3`	`claim_settlement` proven replay-safe in test. This lands *before* the sweep touches production, because Cincinnati is mid-tournament and mostly settled.
4	`Reconciliation · §2.1` PROOF GATES 1–5	Both Cincinnati branches heal from a scheduled run, audited and replay-safe, and a withheld fourth result is caught unprompted.
5	`Database eligibility · §4`	Enum extended, generated column in place, trigger stamps eligibility, insert refused against an ineligible draw, demotion proven non-destructive.
6	`Consumer surface filter · §5`	`public_calendar` live and read by every surface in §5.1, ineligible detail routes 404, CI grep in place, empty state shipped in three locales.
7	`Integrity gate before publication`	No draw becomes playable without a green gate. If it blocks, the tournament stays draw pending and someone is paged — it does not publish anyway.
8	`Lock verified against the clock`	A write attempted one second after `locks_at` is refused by the database, not by a disabled button.

TWO CONSTRAINTS THE ORDER HAS TO RESPECT

The Cincinnati proof window closes with its finals. After that the live stalled data is gone and the reconciliation proof has to be manufactured on Winston-Salem, which is also the rehearsal — one event doing two jobs. **Rows 1 and 2 gate the rehearsal.** Winston-Salem opening on a wrong main-draw date means the rehearsal is testing the wrong lock, which is the one failure that cannot be repaired afterwards.

7.1 Acceptance matrix

GUARANTEE	TEST	ASSERTS
Missing fixture is created	`reconcile.test.ts`	A provider fixture with no stored row is inserted and settled
Repair is replay-safe	`25_reconciliation.test.sql`	Three consecutive sweeps settle once
Orphans are flagged, not deleted	`reconcile.test.ts`	A stored fixture absent from the provider raises, and survives
Lock precedes first ball	`lock.test.ts`	A Monday-anchored US Open fails `assertLockIsSound`
Eligible draw carries a zone	`27_event_dates.test.sql`	No eligible tournament has a null `time_zone`
SQL and TS agree	`23_eligibility.test.sql`	`PUBLIC_TIERS` equals the function's accepted set
Ineligible bracket refused	`23_eligibility.test.sql`	Insert against a Challenger draw fails at the database
Demotion is non-destructive	`23_eligibility.test.sql`	An existing bracket survives, reads and settles
No surface returns anything below 250	`consumer-boundary.test.ts`	Every route in §5.1 returns zero ineligible rows, seeded with Challengers and ITF
Ineligible detail route 404s	`consumer-boundary.test.ts`	A Challenger tournament URL returns not-found, and is absent from the sitemap
League routes survive demotion	`consumer-boundary.test.ts`	A bracket on a demoted tournament still reads at its league-scoped URL
No direct table read	`CI grep`	No `from('tournaments')` in any consumer route, league creation or sitemap
Empty state names the next event	`consumer-boundary.test.ts`	With zero open draws, the response carries the next upcoming event
Override cannot promote	`23_eligibility.test.sql`	`product_override` has no value that makes an ITF event eligible

What this changes in Playbook v2.0

V2.0	STATUS	CHANGE
§2 Priority order	SUPERSEDED	Ten priorities become four before launch. Scope splits into database eligibility and consumer surface filter, which v2.0 treated as one item.
§3 Cut line	SUPERSEDED	Replaced by §7. Dates move forward one day: this directive is dated 21 August, not 20.
§5 Scope	AMENDED	Tier list confirmed as five enum values. Insert-time enforcement added — v2.0 specified the eligibility flag but not the policy that makes it binding at write time.
§7 Reconciliation	AMENDED	The loop must iterate the *provider's* fixture set. v2.0's skeleton compared stored rows against the provider, which cannot find a fixture that was never ingested — the exact shape of the WTA Cincinnati branch.
§11 State	AMENDED	Derivation now reads `main_draw_starts_on` and `locks_at` rather than an ambiguous `starts_on`.
§13 Merchandising	SUPERSEDED	Replaced by §5. v2.0 said the CTA should land on open events first; this directive says ineligible events are not returned at all. Ordering was too weak a guarantee.
§14 Timezone	PROMOTED	Was post-launch. A tournament `time_zone` is now required for any eligible event, because lock validation and displayed times both depend on it.
New	§3	Event-date and lock-time semantics. Absent from v2.0, which flagged only the US Open date conflict without recognising that `starts_on` has no defined meaning.

Everything in v2.0 not listed above stands: §1 preflight, §4.1 cross-cutting rules, §6 tour identity, §8 player display, §9 verification matrix, §10 surface, §12 load, §15 template hygiene, §16 integrity gate, §17 rehearsal gate, §19 posture.

> **THE SENTENCE TO HOLD ON TO**
>
> Right now subtraction is the feature. The public trust surface should be the size the team can defend, and it grows when defending it has stopped being interesting — not before.